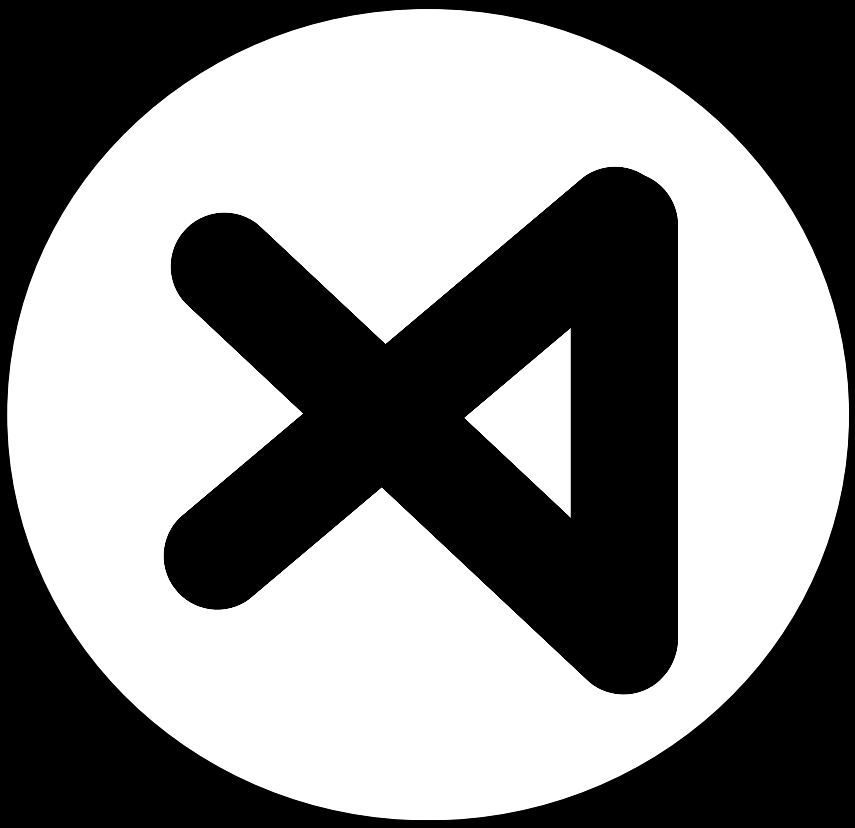


Price Negotiation Strategy Engine

[Alpha-Beta]

Name : Abdelrahman Yasser Shaban

ID : 2401257205

Problem Analysis

Price Negotiation Strategy Engine [Alpha-Beta]

- The project is a negotiation game between the Seller and the buyer with the (Alpha-Beta) Algorithm .

- **Every Players Moves :**

  Seller Moves:

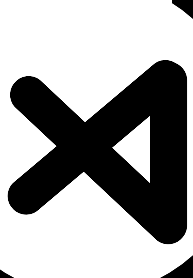
  - Lowers -5
  - Hold
  - Raise +3

  Buyer Moves:

  - Offer +5
  - Hold
  - Walk Away

- **End Conditions**

  if the Buyer Choose “Walk Away”
  Full Rounds Ended
  if The Seller Price <= Buyer Offer , There is a Deal !!



Core Functions

- **Evaluate ()**

  This Function evaluates who is the winner ,and the tree depends on it to Take its Path .

- **Minimax ()**

  The Normal Minimax that we have Studied before through the Labs.

- **Alpha_Beta ()**

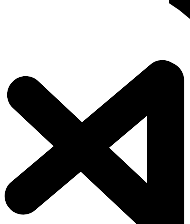
  The Alpha-Beta Algorithm is a Special type of the minimax and it's more efficient than the minimax.

- **get_optimal_path ()**

  This is the most important funcition in the whole project becuase it's the core logic of the program and decides the move of both players and return the result.

- **Compare_Nodes ()**

  This function Compares between the path of Minimax Function and the Alpha-Beta and Shows the difference between them and the pruning percentage.



Source Code

```python
# tkinter Library for GUI
import tkinter as tk
from tkinter import ttk

import math
```

Impots the essential libraries for the program :

- tkinter for the GUI

bultin function ,provides Tree Drawing , Customized buttons.

- math Library

For the values of (-infinity , infinity)

```python
# --------------Main Core Functions----------------

minimax_nodes    = 0
Alpha_Beta_nodes = 0
```

Variables for the Minimax and the Alpha-Beta to Compare between them in the nodes and the Optimal-Path.

```python
def Evaluate(seller_price , buyer_offer) :
    """
    - This Function Compares the Buyer's Offer and the Seller's Price
    - If the Seller's Price is good for the Buyer , He will get a High Score
    - If the Buyer walks away ,it will Return a low Score
    """

    if buyer_offer >= seller_price :
        return 1000 + seller_price   # The 1000 Guarantee a High Score if the Seller wins

    if buyer_offer < 0 :
        return -999999



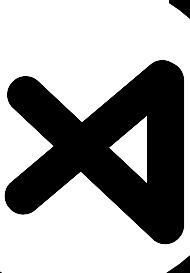
    return -(seller_price - buyer_offer)
```

The Evaluate ( ) takes two parameters :
(seller_price , buyer_offer) and compare between them.

- if the buyers_offer >= the seller_price will return the seller_price with 1000+ Bounus which means They had a deal and the program of negotiation won't run.

- if the buyer_offer < 0 that means He Choose **“Walk Away”** and the program will End.
- if neither of both the program will return the difference between them as a negative value for the seller and run the negotiation game.

The function has a full description also in the code



```python
def minimax (seller_price , buyer_offer , depth , is_seller_turn) :
    global minimax_nodes  # Global Variable to Count the Number of Nodes in the Minimax Tree
    minimax_nodes += 1



    Seller_Move = [-5 , 0 , 3]
    Buyer_Move = [5 , 0 , "WALK AWAY"]



    if depth == 0 : # Stop when the tree is over
        return Evaluate(seller_price , buyer_offer)

    if buyer_offer >= seller_price : # Stop If there is a deal
        return Evaluate(seller_price , buyer_offer)



    if is_seller_turn :
        best =  -math.inf    # Assign the best value to -infinity



        for move in Seller_Move :    # Reassign the seller price in the negotiation
            new_seller_price = seller_price + move
            value = minimax(new_seller_price , buyer_offer , depth-1 , False)
            best = max(best , value)
        return best



    else:
        best = math.inf   # Assign the best value to infinity



        for move in Buyer_Move :
            if move == "WALK AWAY" :
                best = min (best , -999999)
                continue

            new_buyer_offer = buyer_offer + move
            value = minimax(seller_price , new_buyer_offer , depth -1 , True)
            best = min (best , value)
        return best
```

Minimax ( ) :
it's the normal function that we had studied through all the Labs.

There's a global variable in the function and a counter , its job is to count the nodes of the Minimax for every iteration and plus it to the variable to use it later.

There is a list of Bothe players' Moves that I have mentioned before ,that We will use it in the loops and I put it in a list not directly in the loop for more Clean Coding.

It has two Conditions to stop the function :

- if the depth == 0 #which means that Tree is Over
- if the buyer_offer >= seller_price
  #which means "We had a Deal !!"

In the Seller Turn ,the best will = -inf
In the Buyer Turn ,the best will = inf
#To Guarantee that the Seller will be the Maximiser and the Buyer is the Minimiser

If the Buyer Choice was **"Walk Away"** that will make the best is the min (Best , -9999999)
-9999999 represents the value of -inf

```python
def Alpha_Beta(Seller_price , buyer_offer ,depth , alpha , beta , is_seller_turn):
    global Alpha_Beta_nodes  # Global Variable to Count the Number of Nodes in the Alpha-Beta Tree
    Alpha_Beta_nodes += 1



    Seller_Move = [-5 , 0 , 3]
    Buyer_Move = [5 , 0 , "WALK AWAY"]

    if depth == 0 : # Stop when the tree is over
        return Evaluate(Seller_price , buyer_offer)

    if buyer_offer >= Seller_price : # Stop If there is a deal
        return Evaluate(Seller_price , buyer_offer)

    if is_seller_turn :
        best =  -math.inf    # Assign the best value to -infinity



        for move in Seller_Move :    # Reassign the seller price in the negotiation
            new_seller_price = Seller_price + move
            value = Alpha_Beta(new_seller_price , buyer_offer , depth-1 , alpha , beta , False)
            best = max(best , value)
            alpha = max(alpha , best)

            if beta <= alpha :  # Beta Cutting
                break

        return best

    else:
        best = math.inf   # Assign the best value to infinity

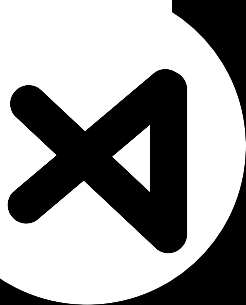


        for move in Buyer_Move :
            if move == "WALK AWAY" :
                best = min (best , -999999)
                beta = min(beta , best)
                if beta <= alpha :  # Alpha Cutting
                    break
                continue

            new_buyer_offer = buyer_offer + move
            value = Alpha_Beta(Seller_price , new_buyer_offer , depth -1 , alpha , beta , True)
            best = min (best , value)
            beta = min(beta , best)

            if beta <= alpha :  # Alpha Cutting
                break

        return best
```



Alpha_Beta ( ) :
it's a special type of the minimax that more efficient than the minimax and don't visit the unnecessary nodes to reduce time and memory.

There's a global variable in the function and a counter , its job is to count the nodes of the Alpha-Beta for every iteration and plus it to the variable to use it later.

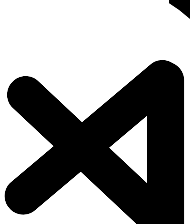
There is a list of Bothe players' Moves that I have mentioned before ,that We will use it in the loops and I put it in a list not directly in the loop for more Clean Coding.

It has two Conditions to stop the function :

- if the depth == 0 #which means that Tree is Over
- if the buyer_offer >= seller_price
  #which means "We had a Deal !!"

The Alpha-Beta has the same logic of the minimax for player turns ,but the difference is that Alpha-Beta has attributes (Alpha - Beta) that used to evaluate the useless nodes and not visit them to reduce time.

The main logic of the Minimax and the Alpha-Beta is the recursion of the function.



```python
def get_optimal_path (seller_price , buyer_offer):
    """
        - This Function Returns the Optimal Path of the Negotiation between the Seller and the Buyer
        - It Uses the Alpha-Beta Pruning Algorithm to Get the Optimal Solution for Each Player
    """

    Buyer_Move = [5 , 0 , "WALK AWAY"]

    optimal_path = []
    alpha = -math.inf
    beta = math.inf

    for round_num in range (1,11) :  # We will have 10 Rounds of Negotiation

        if buyer_offer >= seller_price :
            break

        Seller_Move = [-5 , 0 , 3]
        best_value = -math.inf
        best_move = 0

        for move in Seller_Move :
            new_seller_price = seller_price + move
            value = Alpha_Beta(new_seller_price , buyer_offer , 6 , alpha , beta , False)  # The depth is 6 for 3 Rounds for Each Player

            if value > best_value :
                best_value = value
                best_move = move


        seller_price += best_move
        alpha = max(alpha , best_value)

        if best_move == 0 :
            action = "Hold"

        elif best_move > 0 :
            action = f"Increase Price by {best_move}"

        else :
            action = f"Decrease Price by {best_move}"

        optimal_path.append( # Append the Current State of the Negotiation to the Optimal Path
        {  "Round" : round_num ,"Agent" : "Seller" , "Action" : action , "Seller Price" : seller_price ,
           "Buyer Offer" : buyer_offer , "Alpha" : alpha , "Beta" : beta
        } )

        if buyer_offer >= seller_price :
            break

        best_value  = math.inf
        best_move = 0

        for move in Buyer_Move :
            if move == "WALK AWAY" :
                value = -999999
            else :
                new_buyer_offer = buyer_offer + move
                value = Alpha_Beta(seller_price , new_buyer_offer , 6-round_num , alpha , beta , True)

            if value < best_value :
                  best_value = value
                  best_move  = move

        beta = min(beta , best_value)

        if best_move == "WALK AWAY" :
            optimal_path.append( # Append the Current State of the Negotiation to the Optimal Path
                { "Round" : round_num ,"Agent" : "Buyer" , "Action" : "WALK AWAY" , "Seller Price" : seller_price ,
                  "Buyer Offer" : buyer_offer , "Alpha" : alpha , "Beta" : beta })
            return optimal_path , "The Buyer Walked Away !!" , None

        buyer_offer += best_move

        action = "Hold" if best_move == 0 else f"Increase Offer by {best_move}"

        optimal_path.append(# Append the Current State of the Negotiation to the Optimal Path
            { "Round" : round_num , "Agent" : "Buyer" , "Action" : action , "Seller Price" : seller_price ,
              "Buyer Offer" : buyer_offer , "Alpha" : alpha , "Beta" : beta })

    if buyer_offer >= seller_price :
        return optimal_path , "We Had A Deal !!" , seller_price

    return optimal_path , f"No Deal After {round_num - 1} Rounds" , None
```

get_optimal_path ( ) :
This function is the most important one in the whole program because of its role for decide which node in the tree the player will choose.

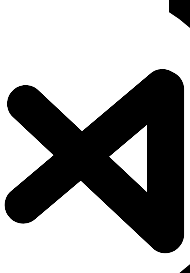
The function has a full description also in the source code.

We have a list that called **optimal_path [ ]** to store the path and return it.

We have a for loop that have range **(1:11)** to garantee looping for 10 rounds.
The last value in the Range is excluded

The main logic of the function is to compare the move of the player with **min( )** and **max( )** functions and store it in the alpha variable for the Seller and the beta variable for the Buyer.

We have an important command in the function ,that is to append the result of the iteration in the optimal_path list as a **Dictionary Data Type** to use it later for the **GUI**.



```python
def compare_nodes (seller_price , buyer_offer ) : # This Function Compares Between Minimax and Alpha-Beta
    global minimax_nodes
    global Alpha_Beta_nodes

    minimax_nodes = 0
    Alpha_Beta_nodes = 0

    minimax(seller_price , buyer_offer , 6 , True)
    Alpha_Beta(seller_price , buyer_offer , 6 , -math.inf , math.inf , True)

    pruning_percentage = round ((1 - Alpha_Beta_nodes / minimax_nodes) * 100 ,2 )

    return  {
        "Minimax Nodes" : minimax_nodes ,
        "Alpha-Beta Nodes" : Alpha_Beta_nodes ,
        "Pruning Percentage" : f"{pruning_percentage} %"
    }
```

compare_nodes ( ) :
This function compares between the **Minimax ( )** and the **Alpha_Beta ( )** and call both of them with depth of 6 Rounds # As it required

This functions has the counter of the Minimax nodes and the Alpha-Beta nodes to compare them.

The Pruning Percentge as an equation :
(1- Alpha_Beta_nodes / minimax_nodes) * 100
to evaluate the ratio between both of the functions.

GUI

Graphical User Interface

```
#------------------GUI------------------

# ── Main Window ──
root = tk.Tk()
root.title("Price Negotiation Strategy Engine")
root.geometry("900x700")
root.configure(bg="#1e1e2e")

```

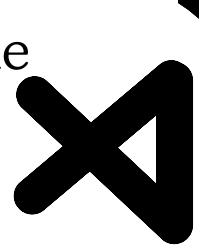
The main Window :

The function defines the aspects of the windows and make it with dark background.

```
# ── Title ──
title_label = tk.Label(
    root,
    text="Price Negotiation Strategy Engine",
    font=("Arial", 18, "bold"),
    bg="#1e1e2e",
    fg="#cdd6f4"
)
title_label.pack(pady=20)
```

The Title :

The function defines the main title at the top of the windows with the project title.



```python
# ── Input Frame ──
input_frame = tk.Frame(root, bg="#313244", padx=20, pady=20)
input_frame.pack(fill="x", padx=30, pady=10)

# Seller Price
seller_label = tk.Label(
    input_frame,
    text="Seller Starting Price :",
    font=("Arial", 12),
    bg="#313244",
    fg="#cdd6f4"
)
seller_label.grid(row=0, column=0, padx=10, pady=10, sticky="w")

seller_entry = tk.Entry(
    input_frame,
    font=("Arial", 12),
    width=10,
    bg="#45475a",
    fg="#cdd6f4",
    insertbackground="#cdd6f4"
)
seller_entry.insert(0, "100")
seller_entry.grid(row=0, column=1, padx=10, pady=10)
```

The Seller Price :

The function defines an input frame for the user input of the Negotiation Values for the Seller

The Frames has an initial value for the Seller as 100.



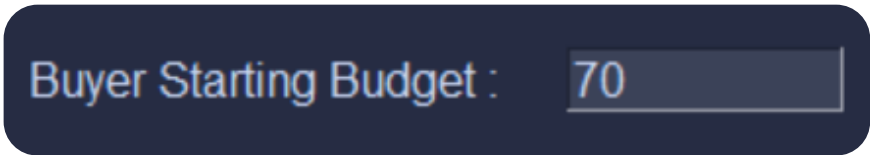
```python
# Buyer Budget
buyer_label = tk.Label(
    input_frame,
    text="Buyer Starting Budget :",
    font=("Arial", 12),
    bg="#313244",
    fg="#cdd6f4"
)
buyer_label.grid(row=0, column=2, padx=10, pady=10, sticky="w")

buyer_entry = tk.Entry(
    input_frame,
    font=("Arial", 12),
    width=10,
    bg="#45475a",
    fg="#cdd6f4",
    insertbackground="#cdd6f4"
)
buyer_entry.insert(0, "70")
buyer_entry.grid(row=0, column=3, padx=10, pady=10)
```

The Buyer Price :

The function defines an input frame for the user input of the Negotiation Values for the Buyer.

The Frames has an initial value for the Buyer as 70.



```python
# Run Button
run_button = tk.Button(
    input_frame,
    text="Run Negotiation",
    font=("Arial", 12, "bold"),
    bg="#89b4fa",
    fg="#1e1e2e",
    padx=15,
    pady=5,
    cursor="hand2",
    command=lambda: run_negotiation()
)
run_button.grid(row=0, column=4, padx=20, pady=10)
```

The Run Button :

The function designs the run button to trigger the command of lambda to run the Function **run_negotiation** and start the core functions of the program.



```python
# ── Notebook (Tabs) ──
notebook = ttk.Notebook(root)
notebook.pack(fill="x", expand=False, padx=30, pady=10)  # ← expand=False و fill="x"

# Tab 1 — Results Table
tab1 = tk.Frame(notebook, bg="#313244", height=200)  # ← height ثابت
notebook.add(tab1, text="  Negotiation Results  ")

results_label = tk.Label(
    tab1,
    text="Negotiation Results",
    font=("Arial", 13, "bold"),
    bg="#313244",
    fg="#cdd6f4"
)
results_label.pack(anchor="w", pady=5, padx=10)

columns = ("Round", "Agent", "Action", "Seller Price", "Buyer Offer", "Alpha", "Beta")
tree = ttk.Treeview(tab1, columns=columns, show="headings", height=6)
for col in columns:
    tree.heading(col, text=col)
    tree.column(col, anchor="center", width=120)
tree.pack(fill="both", expand=True, padx=10, pady=5)
```

The Notebook :

It defines the tabs of the window.

This is the code of Tab1 that displays the Steps of the negotiation as a table of columns :

- Round Number
- Player “Agent”
- Action
- Seller Price
- Buyer Offer
- Alpha and Beta Values



```python
# Tab 2 — Negotiation Tree
tab2 = tk.Frame(notebook, bg="#1e1e2e", height=200)
notebook.add(tab2, text="  Negotiation Tree  ")

# ── Scrollbars ──
scroll_y = tk.Scrollbar(tab2, orient="vertical")
scroll_x = tk.Scrollbar(tab2, orient="horizontal")
scroll_y.pack(side="right", fill="y")
scroll_x.pack(side="bottom", fill="x")

tree_canvas = tk.Canvas(
    tab2,
    bg="#1e1e2e",
    highlightthickness=0,
    height=200,
    yscrollcommand=scroll_y.set,
    xscrollcommand=scroll_x.set
)
tree_canvas.pack(fill="both", expand=True)

scroll_y.config(command=tree_canvas.yview)
scroll_x.config(command=tree_canvas.xview)
```

The Notebook :

This the code of **Tab2** that display the negotiation tree with the pruned nodes with Gray Color.

The tab with Scroll Slider to See The Whole tree ,with key values on the side of the window to make the tree easier to read



```python
# ── Outcome Label ──
outcome_label = tk.Label(
    root,
    text="",
    font=("Arial", 13, "bold"),
    bg="#1e1e2e",
    fg="#a6e3a1"
)
outcome_label.pack(pady=8)

# ── Stats Frame ──
stats_frame = tk.Frame(root, bg="#313244", padx=20, pady=15)
stats_frame.pack(fill="x", padx=30, pady=10)

minimax_label   = tk.Label(stats_frame, text="Minimax Nodes  : ", font=("Arial", 11), bg="#313244", fg="#cdd6f4")
alphabeta_label = tk.Label(stats_frame, text="Alpha-Beta Nodes : ", font=("Arial", 11), bg="#313244", fg="#cdd6f4")
pruning_label   = tk.Label(stats_frame, text="Pruning %  : ", font=("Arial", 11), bg="#313244", fg="#cdd6f4")

minimax_label.grid  (row=0, column=0, padx=20, pady=5)
alphabeta_label.grid(row=0, column=1, padx=20, pady=5)
pruning_label.grid  (row=0, column=2, padx=20, pady=5)
```

The Outcome Lable :

This function for the Stats of the negotiation :

- Number of Minimax's Nodes
- Number of Alpha-Beta's Nodes
- The Pruning percentage
- The Final Result of the negotiation (Deal , No Deal)



```python
def draw_tree(seller_price, buyer_offer):
    tree_canvas.delete("all")
    tree_canvas.config(scrollregion=(0, 0, 1200, 600))  # ← كبّر المساحة

    Seller_Move = [-5, 0, 3]
    Buyer_Move  = [5, 0, "WALK AWAY"]

    W      = 900
    node_r = 32

    def draw_node(x, y, text1, text2, color, pruned=False):
        fill    = "#555566" if pruned else color
        outline = "#888899" if pruned else "#ffffff"
        tc      = "#888899" if pruned else "#ffffff"
        tree_canvas.create_oval(x-node_r, y-node_r, x+node_r, y+node_r,
                                fill=fill, outline=outline, width=1.5)
        tree_canvas.create_text(x, y-9, text=text1, fill=tc,
                                font=("Arial", 9, "bold"))
        tree_canvas.create_text(x, y+9, text=text2, fill=tc,
                                font=("Arial", 8))

    def draw_edge(x1, y1, x2, y2, label, pruned=False):
        color = "#555566" if pruned else "#89b4fa"
        dash  = (5, 4)   if pruned else ()
        tree_canvas.create_line(x1, y1+node_r, x2, y2-node_r,
                                fill=color, width=1.5, dash=dash)
        mx = (x1+x2)/2
        my = (y1+y2)/2 - 10
        tree_canvas.create_text(mx, my, text=label,
                                fill="#a6adc8" if not pruned else "#555566",
                                font=("Arial", 8))

    # ── Level 0: Root ──
    rx, ry = W//2, 60
    draw_node(rx, ry, f"S:{seller_price}", f"B:{buyer_offer}", "#89b4fa")

    # ── Level 1: Seller moves ──
    l1_xs  = [rx-310, rx, rx+310]
    l1_y   = 200
    s_lbls = ["-5", "Hold", "+3"]
    alpha  = -math.inf

    for i, (sm, lx) in enumerate(zip(Seller_Move, l1_xs)):
        nsp   = seller_price + sm
        score = Alpha_Beta(nsp, buyer_offer, 4, -math.inf, math.inf, False)

        draw_edge(rx, ry, lx, l1_y, s_lbls[i])
        s_txt = f"{round(score,0)}" if score > -999990 else "WA"
        draw_node(lx, l1_y, f"S:{nsp}", s_txt, "#cba6f7")

        alpha = max(alpha, score)

        # ── Level 2: Buyer moves ──
        l2_xs  = [lx-95, lx, lx+95]
        l2_y   = 370
        b_lbls = ["+5", "Hold", "WA"]
        beta   = math.inf
        pruned_b = False

        for j, (bm, bx) in enumerate(zip(Buyer_Move, l2_xs)):
            if bm == "WALK AWAY":
                bs    = -999999
                btxt1 = "WALK"
                btxt2 = "AWAY"
            else:
                nbo   = buyer_offer + bm
                bs    = Alpha_Beta(nsp, nbo, 3, -math.inf, math.inf, True)
                btxt1 = f"B:{nbo}"
                btxt2 = f"{round(bs,0)}" if bs > -999990 else "WA"

            draw_edge(lx, l1_y, bx, l2_y, b_lbls[j], pruned_b)
            draw_node(bx, l2_y, btxt1, btxt2, "#a6e3a1", pruned_b)

            if not pruned_b:
                beta = min(beta, bs)
                if beta <= alpha:
                    pruned_b = True
```

Draw_Tree( ) :
This function for Drawing the Tree of the negotiation with players moves as a possible nodes ,levels from Root to the Top and Display the same path of the **get_optimal_path ( ).**

```python
# ── Run Negotiation Function ──
def run_negotiation():
    try:
        seller_price = int(seller_entry.get())
        buyer_offer  = int(buyer_entry.get())
    except ValueError:
        outcome_label.config(text="Please enter valid numbers!", fg="#f38ba8")
        return


    # امسح الجدول القديم
    for row in tree.get_children():
        tree.delete(row)

    # هشل الـ logic
    path, outcome, final_price = get_optimal_path(seller_price, buyer_offer)
    stats = compare_nodes(seller_price, buyer_offer)

    # املا الجدول
    for step in path:
      tree.insert("", "end", values=(
         step["Round"],
         step.get("Agent", "—"),
         step["Action"],
         step["Seller Price"],
         step["Buyer Offer"],
         format_val(step["Alpha"]),
         format_val(step["Beta"])
    ))

    # اعرض النتيجة
    if outcome == "We Had A Deal !!":
        outcome_label.config(
            text=f"Deal at Price: {final_price}",
            fg="#a6e3a1"
        )
    else:
        outcome_label.config(
            text=outcome,
            fg="#f38ba8"
        )

   # ارسم الشجرة
    draw_tree(seller_price, buyer_offer)

    # اعرض الـ stats
    minimax_label.config  (text=f"Minimax Nodes   : {stats['Minimax Nodes']}")
    alphabeta_label.config(text=f"Alpha-Beta Nodes : {stats['Alpha-Beta Nodes']}")
    pruning_label.config  (text=f"Pruning %       : {stats['Pruning Percentage']}")


root.mainloop()
```

output

Price Negotiation Strategy Engine

Seller Starting Price : 100 Buyer Starting Budget : 70 Run Negotiation

Negotiation Results | Negotiation Tree

Negotiation Results

Round	Agent	Action	Seller Price	Buyer Offer	Alpha	Beta
1	Seller	Decrease Price by -5	95	70	-inf	Deal
1	Buyer	Increase Offer by 5	95	75	-inf	-inf
2	Seller	Increase Price by 3	98	75	Deal	-inf
2	Buyer	WALK AWAY	98	75	Deal	-inf

The Buyer Walked Away !!

Minimax Nodes : 388 Alpha-Beta Nodes : 120 Pruning % : 69.07 %

Price Negotiation Strategy Engine

Seller Starting Price : 90 Buyer Starting Budget : 80 Run Negotiation

Negotiation Results | Negotiation Tree

Negotiation Results

Round	Agent	Action	Seller Price	Buyer Offer	Alpha	Beta
1	Seller	Decrease Price by -5	85	80	-inf	Deal
1	Buyer	WALK AWAY	85	80	-inf	-inf

The Buyer Walked Away !!

Minimax Nodes : 231 Alpha-Beta Nodes : 132 Pruning % : 42.86 %

Price Negotiation Strategy Engine

Seller Starting Price : 85 Buyer Starting Budget : 80 Run Negotiation

Negotiation Results | Negotiation Tree

Negotiation Results

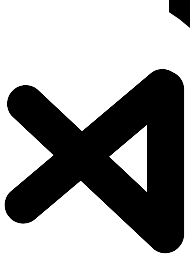
Round	Agent	Action	Seller Price	Buyer Offer	Alpha	Beta
1	Seller	Decrease Price by -5	80	80	Deal	Deal

Deal at Price: 80

Minimax Nodes : 118 Alpha-Beta Nodes : 90 Pruning % : 23.73 %

Summury Table

Attributes	1st - Run	2nd - Run	3rd - Run
Seller Price	**100**	**90**	**100**
Buyer Offer	**70**	**85**	**120**
Minimax Nodes	388	118	1
Alpha Beta Nodes	120	90	1
Pruning	69.07%	23.73%	0.0 %
Final Outcome	**Buyer Walked Away**	**Deal at Price : 85**	**Deal at Price : 100**



From the most dark side of the world

“Intelligent people Can act lazy because they will find the most efficient way to accomplish tasks”

Linus Torvalds

With great Regards

A.YASSER *Phoenix*